МИРЭА
Российский технологический университет

Захват файлов и записей

Мухортова Н.Н.

Цель

Познакомиться с инструментарием управления файлами и записями в файлах. Освоить инструментарий управления.

Захват записей

Захват (блокировка) записей в ОС UNIX - это базовое средство синхронизации процессов, осуществляющих доступ к одной и той же записи в файле, а не средство обеспечения безопасности.

Захват записи осуществляется системным вызовом fcntl(2).

Запись

Запись (record) - это последовательный набор байтов в файле. У записи есть начальная позиция в файле и длина, отсчитываемая от этой позиции. Начало и длина указываются с точностью до байта. Ядро Unix не делает никаких предположений о том, как эта запись интерпретируется прикладной программой.

Одна запись с точки зрения Unix, с точки зрения приложения может представлять собой одну или несколько записей базы данных или часть такой записи или текстовую строку или что-то ещё. Если захваченная запись начинается с начала файла и равна файлу по длине, иногда говорят о захвате всего файла.

Захват записи по чтению

Захват записи по чтению (разделяемый доступ) не даёт другим процессам установить захват записи по изменению, но позволяет неограниченному количеству процессов устанавливать захваты по чтению.

Захват записи по изменению

Захват записи по изменению (эксклюзивный доступ) не даёт другим процессам установить захват записи ни по чтению, ни по изменению, пока этот захват по изменению не будет снят.

Конфликтующие захваты

Конфликтующие захваты — пара захватов, несовместимых друг с другом (два захвата по изменению или один захват по чтению и один по изменению) и при этом устанавливаемых на перекрывающиеся участки файла. Перекрытия на один байт достаточно для возникновения конфликта.

Рекомендательный режим захвата

Рекомендательный (advisory) режим захвата не взаимодействует с подсистемой ввода/вывода. Это означает, что захват проверяется только при попытках захвата, но не при операциях чтения и записи. Как и семафоры и мьютексы при работе с памятью, рекомендательный захват указывает другим процессам, что им не следует работать с захваченным участком, но не ставит физических препятствий.

Обязательный режим захвата

Обязательный (mandatory) режим захвата взаимодействует с подсистемой ввода/вывода. Это означает, что системные вызовы read(2) или write(2) будут приостанавливаться (неявный захват), пока не будет снят захват соответствующей записи.

Тип захвата

Тип захвата (по чтению или по модификации) определяется параметрами fcntl(2), точнее, полями в структуре flock. Режим захвата (рекомендательный или обязательный) определяется атрибутами файла.

По умолчанию, в Unix используется рекомендательный захват.

Чтобы установить обязательный захват, файлу необходимо установить бит setgid, но при этом файл не должен быть доступен по исполнению для группы. Все захваты, устанавливаемые на такой файл, будут работать в обязательном режиме

Взаимные блокировки

Ядро обеспечивает определенную степень обнаружения и предотвращения взаимных блокировок (deadlock). При установке захвата осуществляется проверка, что не создается цикл процессов, каждый из которых ждет от других снятия захвата записи/файла. Если такая ситуация обнаружена, fcntl(2) немедленно возвращает неуспех и устанавливает errno равным EDEADLK.

Установка и снятие захвата

Системный вызов fcntl(2) используется для захвата файла и записи и для снятия захвата. Его аргументы:

fildes дескриптор файла, обычно получают с помощью системного вызова open(2)

cmd определяет одну из трех команд, на след. слайде

arg адрес структуры захвата struct flock

cmd команды

F_SETLK Эта команда используется, чтобы установить или снять захват записи. Структура захвата используется для задания расположения записи, ее длины и типа захвата. Если захват нельзя установить из-за конфликтующего захвата или по какой-то другой причине, fcntl вернет -1 и установит errno.

F_SETLKW Эта команда аналогична F_SETLK, кроме того что, если на указанный участок установлена конфликтующая блокировка, fcntl приостановится, пока запись не освободится.

F_GETLK Эта команда используется, чтобы получить информацию о захвате записи. Чтобы определить расположение записи и тип захвата используется структура захвата flock.

F_GETLK выдает информацию о первом захвате записи, перекрывающейся с участком файла, описанным в структуре flock. Возвращаемая информация затирает информацию, переданную вызову fcntl через эту структуру. Если нет конфликтующих захватов, структура возвращается назад неизмененной, кроме типа захвата, который устанавливается равным F_UNLCK.

F_GETLK возвращает какой-то из захватов, который был установлен на некоторый момент времени. Это означает, что при интерпретации информации, полученной через F_GETLK надо учитывать, что с момента вызова F_GETLK на файл могли быть установлены новые блокировки или сняты какие-то из существовавших, в том числе и та блокировка, информацию о которой вернул этот вызов.

fcntl(..) или lockf(..)

Как при рекомендательном, так и при принудительном захвате записи, захват выполняется с использованием fcntl(2) или lockf(3C). Различие между двумя формами захвата состоит в том, когда он проверяется.

При принудительном захватывании он проверяется перед каждой операцией ввода/вывода.

При рекомендательном захватывании он проверяется, когда делается попытка захвата с помощью fcntl(2) или lockf(3). Захваты, которые не сняты явно с использованием fcntl(2), снимаются при завершении процесса с помощью exit(2) или при закрытии файла с помощью close(2)

Захват записи

Чтобы выполнить захват записи, ваша программа должна объявить переменную типа struct flock или выделить память под такую структуру другим способом (например, через malloc(3C)), присвоить значение полям этой структуры и передать её адрес системному вызову fcntl в качестве третьего аргумента.

Поля struct flock

l_type указывает тип захвата. Его возможные значения:

- F_RDLCK захват по чтению
- F_WRLCK захват по изменению
- F_UNLCK снятие захвата. Только процесс, который захватил запись, может освободить её. Освобождение сегмента из середины большой записи оставляет два захваченных сегмента с двух концов.

Поля struct flock

l_whence является признаком относительной стартовой позиции записи:

- SEEK_SET (0) - от начала файла,
- SEEK_CUR (1) - от текущей позиции в файле и
- SEEK_END (2) - от конца файла. Это поле аналогично аргументу whence в системном вызове lseek(2).

Поля struct flock

l_start определяет начальную позицию записи в зависимости от l_whence.

Это поле аналогично аргументу offset вызова lseek(2).

При l_whence, равном SEEK_CUR или SEEK_END, l_start может быть отрицательным.

Поля struct flock

l_len определяет длину захватываемой записи. Значение ноль захватывает/освобождает от начальной позиции до конца файла.

Если после такого захвата длина файла увеличится, все новые данные, появляющиеся в файле, также будут захвачены.

Поля struct flock

l_pid устанавливается операционной системой равным идентификатору процесса, который захватил запись, когда выполняется запрос F_GETLK.

При установке захвата не используется.

l_sysid устанавливается при запросе F_GETLK равным RFS-идентификатору удалённой системы, на которой размещён этот файл.

Печать отчёта - Пример

Этот пример в этом разделе читает и изменяет файл данных о служащих.

Захваты по записи и изменению используются, чтобы запретить нескольким процессам изменять одну и ту же запись в одно и то же время.

Запись о служащем

employee.h:

Она содержит имя служащего, его оклад и идентификатор пользователя, который последним изменил эту запись.

```
1 #define   NAMESIZE   24
2
3 struct employee {
4     char name[NAMESIZE];
5     int salary;
6     unsigned short uid;
7 };
```

Эта программа выводит содержание файла служащих. Весь файл захватывается по чтению, чтобы не допустить изменение любой записи при печатании отчёта.

Это гарантирует, что отчёт будет точным отражением текущего состояния файла служащих. Захват всего файла не причинит неудобства, если отчёт печатается тогда, когда файл служащих не используется никем другим.

Код, показанный на следующем слайде, содержит только захват и освобождение записи.

Программа работает следующим образом: ... Файл служащих, полученный как первый аргумент командной строки, открывается для чтения.

18-22 Полям структуры захвата присваиваются значения, соответствующие выполнению захвата по чтению.

23-33 Цикл пытается пять раз захватить файл. fcntl завершается неудачно, если где-либо в файле уже есть захват по записи. Если программа так и не смогла захватить файл, печатается сообщение об ошибке, рекомендующее пользователю исполнить программу позже. ... Отсутствующий код печатает записи о служащих, а также вычисляет и распечатывает суммарный оклад.

42-44 Перед завершением программы захват по чтению снимается. В данном случае это не обязательно, ведь захват будет неявно снят при exit(2). Но это может оказаться полезно, если затем в программу будет добавлена какая-то другая функциональность, исполняемая после завершения работы с базой данных

Включения

```
1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4 #include <stdio.h>
5 #include <fcntl.h>
6 #include <errno.h>
7 #include "employee.h"
8 #define   MAXTRIES   5
```

```
 9 main(int argc, char *argv[])
10 {
11     struct flock lock;
...
18     lock.l_type = F_RDLCK;
19     lock.l_whence = SEEK_SET;
20     lock.l_start = 0;
21     lock.l_len = 0; /* whole file address space */
22     while (fcntl(fd, F_SETLK, &lock) == -1) {
23         if ((errno == EACCES) || (errno == EAGAIN)) {
24             if (try++ < MAXTRIES) {
25                 sleep(1);
26                 continue;
27             }
28             printf("%s busy -- try later\n",
29                 argv[1]);
30             exit(2);
31         }
32         perror(argv[1]); exit(3);
33     }
...
42     lock.l_type = F_UNLCK;    /* unlock file */
43     fcntl(fd, F_SETLK, &lock);
44     close(fd);
```

Изменение записи - Пример

В этом примере изменяется запись о служащем. Перед чтением запись захватывается по изменению и освобождается после изменения. Программа работает следующим образом:

Файл служащих открывается для чтения и записи.

После входа в цикл for, программа просит пользователя ввести номер записи.

Изменение записи - Пример

Эта схема захвата записи работает правильно, если все процессы используют эту программу для изменения файла служащих. Этого можно достигнуть соответствующей установкой прав доступа файла служащих и бита "установки идентификатора пользователя" этой программы. Кроме того, обеспечение такого доступа к файлу служащих делает программу уверенной в том, что запись доступна в каждый момент только одному процессу

28-36 Запись захватывается по изменению. Это не дает возможности другим процессам захватить эту запись. fcntl(2) подвиснет, если запись уже захвачена, ожидая пока все другие процессы освободят ее.

38-44 Текущая позиция файла устанавливается на захватываемую запись. Затем запись читается. Замечание: lseek(2) используется явно, чтобы установить текущую позицию в файле. Использование fcntl(2) для захвата записи не приводит к изменению текущей позиции файла. ... Печатается имя и оклад служащего. Затем вводится новый оклад.

51-52 lseek(2) используется снова, чтобы установить позицию файла на захваченную запись. Измененная запись пишется в файл. 54-55 Захват по изменению снимается, чтобы все остальные процессы могли использовать измененную запись.

```
 9 main(int argc, char *argv[])
10 {
11     struct flock lock;
...
28     position = (recnum-1) * sizeof(record);
29     lock.l_type = F_WRLCK;        /* lock record */
30     lock.l_whence = SEEK_SET;
31     lock.l_start = position;
32     lock.l_len = sizeof(record);
33     if (fcntl(fd, F_SETLKW, &lock) == -1) {
34         perror(argv[1]);
35         exit(2);
36     }
37
38     lseek(fd, position, SEEK_SET);      /* read record */
39     if (read(fd, &record, sizeof(record)) == 0) {
40         printf("record %d not found\n", recnum);
41         lock.l_type = F_UNLCK;
42         fcntl(fd, F_SETLK, &lock);
43         continue;
44     }
...
51     lseek(fd, position, SEEK_SET);
52     write(fd, &record, sizeof(record));
53
54     lock.l_type = F_UNLCK;        /* release record */
55     fcntl(fd, F_SETLK, &lock);
...
58 }
```

Изменение записи - Пример отображения на память и захвата записи

Файл сначала отображается на память с помощью mmap(2). Каждая запись перед чтением захватывается по изменению и освобождается после изменения.

Файл служащих открывается для чтения и записи.

23-25 Файл служащих отображается на память. ... После входа в цикл for, программа запрашивает у пользователя номер записи.

34-38 Проверка номера записи.

40-47 Запись захватывается по изменению. Это не дает возможности другим процессам тоже захватить эту запись. fcntl(2) будет заблокирована, если запись уже захвачена, в ожидании пока все остальные процессы не освободят её. ... Печатается имя и оклад служащего. Потом вводится новый оклад.

55-59 Содержимое памяти синхронизуется с файлом вызовом msync(3C). Затем захват по изменению записи снимается, и она становится доступной для других процессов.

```
10 main(int argc, char *argv[])
11 {
12    struct flock lock;
13    off_t size;
14    struct employee *p;
...
23    size = lseek(fd, 0, SEEK_END);
24    p = (struct employee *)mmap(0, size,
25       PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
...
34       position = recnum * sizeof(struct employee);
35       if (position >= size) {
36       printf("record %d not found\n", recnum);
37       continue;
38       }
```

```
39
40      lock.l_type = F_WRLCK;        /* lock record */
41      lock.l_whence = SEEK_SET;
42      lock.l_start = position;
43      lock.l_len = sizeof(struct employee);
44      if (fcntl(fd, F_SETLKW, &lock) == -1) {
45       perror(argv[1]);
46       exit(2);
47      }
...
55      msync(p, size, MS_ASYNC);
56
57      lock.l_type = F_UNLCK;        /* release record */
58      fcntl(fd, F_SETLK, &lock);

...

63 }
```

Изменение записи - Пример

Эта программа является развитием предыдущего примера. После захвата по чтению записи о служащем, информация из неё выводится для пользователя. Потом программа спрашивает пользователя, хочет ли он изменить запись. В случае положительного ответа, захват по чтению переводится в захват по изменению. Потом производится изменение, после чего запись освобождается.

Файл служащих открывается для чтения и записи. После входа в цикл for вводится номер записи.

28-36 Запись захватывается по чтению. Процесс подвешивается, если запись уже захвачена по изменению. ... Если попытка прочитать запись неуспешна, то захват по чтению снимается и программа идет на начало цикла. Иначе печатается имя служащего и его оклад.

50-54 Если пользователь не хочет изменять запись, захват по чтению снимается и программа продолжается с начала цикла. Предупреждение: Не забудьте освободить запись перед переходом на начало цикла.

55-59 Захват по чтению переводится в захват по изменению. fcntl(2) блокируется, если есть другие процессы, захватившие эту запись по чтению. После того как все процессы освободят эту запись, наш процесс просыпается и захватывает запись. ... Запись о служащем изменяется.

66-67 Захват по изменению снимается с записи

```
 9 main(int argc, char *argv[])
10 {
11      struct flock lock;
...
28      position = (recnum-1) * sizeof(record);
29      lock.l_type = F_RDLCK;          /* read lock */
30      lock.l_whence = SEEK_SET;
31      lock.l_start = position;
32      lock.l_len = sizeof(record);
33      if (fcntl(fd, F_SETLKW, &lock) == -1) {
34          perror(argv[1]);
35          exit(2);
36      }
...
50      if (ans[0] != 'y') {
51          lock.l_type = F_UNLCK;
52          fcntl(fd, F_SETLK, &lock);
53          continue;
54      }
```

```
55      lock.l_type = F_WRLCK;          /* write lock */
56      if (fcntl(fd, F_SETLKW, &lock) == -1) {
57          perror(argv[1]);
58          exit(3);
59      }
...
66      lock.l_type = F_UNLCK; /* release record */
67      fcntl(fd, F_SETLK, &lock);
...
70 }
```

Выдача информации о захватах записи - Пример

Так как несколько пользователей могут изменять различные записи в файле, может быть необходимо выдать отчет о захваченных записях. Например, полезно знать, что какие-то записи в файле захватываются слишком часто в одной и той же программе. Тогда эту программу можно исправить.

Выдача информации о захватах записи - Пример

Программа печатает список всех захваченных записей, тип захвата и идентификатор процесса, захватившего запись.

Программа проанализирует на захват все записи в файле. Для этого используется системный вызов fcntl(2) с запросом F_GETLK. Тип захвата должен быть равен F_WRLCK, потому что такой запрос конфликтует со всеми другими типами запросов и, таким образом выдаст информацию обо всех установленных захватах. Тип F_RDLCK конфликтует только с захватами на запись и выдаст информацию только о них.

12 Описание переменной типа struct flock. ... Файл открывается для чтения и определяется его размер при помощи lseek(2).

22 Записи проверяются с начала файла.

25-26 Этот цикл for проверяет каждую запись в файле.

27-30 Тип захвата запроса – это захват по изменению, но fcntl(2) при возврате изменяет поле l_type на тип захвата, установленного на проверяемой записи. Поле длины записи (l_len) и ее начальная позиция (l_start) также могут быть изменены. Именно поэтому l_start используется как переменная цикла в операторе for. Если запись, задаваемая l_start и l_len, свободна, цикл продолжается со следующей записи.

32-33 Печатается тип захвата.

34-35 Печатается номер записи. Заметим, что программа предполагает, что все записи одного и того же размера.

36-39 Если конец файла, начиная с l_start, захвачен, то l_len станет равным нулю. Тогда вычисляется число оставшихся в файле записей.

40-41 Печатается число захваченных записей.

Помните, что l_start и l_len могут измениться при вызове fcntl(2) с F_GETLK. Это происходит, когда расположение существующего захвата или не точно соответствует значениям, переданным fcntl(2) в структуре flock.

```
 9 main(int argc, char *argv[])
10 {
11     struct employee record;
12     struct flock lock;
13     off_t size;
...
22     lock.l_whence = SEEK_SET;
23     lock.l_len = sizeof(record);
24     printf("File: %s\n", argv[1]);
25     for (lock.l_start = 0; lock.l_start < size;
26      lock.l_start += lock.l_len) {
27      lock.l_type = F_WRLCK;
28      fcntl(fd, F_GETLK, &lock);
29      if (lock.l_type == F_UNLCK)     /* not locked */
30          continue;
31      printf("Pid: %ld", lock.l_pid);
32      printf("\t%s lock",
33          (lock.l_type == F_WRLCK)? "Write" : "Read");
34      printf("\tRecord number: %ld",
35          lock.l_start/sizeof(record));
36      if (lock.l_len == 0) {     /* rest of file */
37          lock.l_len = size - lock.l_start;
38          lock.l_start = size;  /* terminate for loop */
39          }
40      printf("\tNumber of records: %ld\n",
41          lock.l_len/sizeof(record));
42     }
43 }
```

Выводы

Часть данных в файле, или весь файл могут быть захвачены с целью не дать возможность поменять информацию или прочитать информацию.

Это необходимо в случае, когда с одним файлом могут работать несколько процессов.